



Institut Universitaire des sciences

- Faculté des sciences et de technologie

✓ TD N°2 Modélisation

Nom & Prénom : COFFY Cliford

Niveau : L3

Prof. : Ismael SAINT AMOUR

Date : 30/03/2026

TD 1: Créer une application GUI qui inclut :

- générateur de mots de passe
- générateur de nombres premiers

```
modelisation.py U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

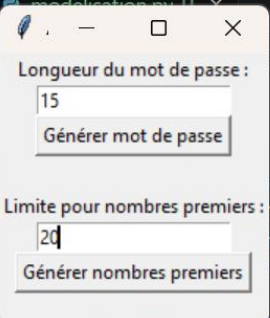
1  import tkinter as tk
2  import string
3  import secrets
4
5  # Fonction pour générer un mot de passe
6  def generate_password():
7      length = int(entry_length.get())
8      charset = string.ascii_letters + string.digits + string.punctuation
9      password = ''.join(secrets.choice(charset) for _ in range(length))
10     result_password.set(password)
11
12     # Fonction pour générer des nombres premiers
13     def generate_primes():
14         limit = int(entry_limit.get())
15         primes = []
16         for num in range(2, limit + 1):
17             is_prime = True
18             for i in range(2, int(num**0.5) + 1):
19                 if num % i == 0:
20                     is_prime = False
21                     break
22             if is_prime:
23                 primes.append(num)
24         result_primes.set(", ".join(map(str, primes)))
25
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

modelisation.py U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > generate_password
13 def generate_primes():
17     is_prime = True
18     for i in range(2, int(num**0.5) + 1):
19         if num % i == 0:
20             is_prime = False
21             break
22     if is_prime:
23         primes.append(num)
24     result_primes.set(", ".join(map(str, primes)))
25
26 # Création de la fenêtre principale
27 root = tk.Tk()
28 root.title("Application Sécurité - TD1")
29
30 # Variables pour afficher les résultats
31 result_password = tk.StringVar()
32 result_primes = tk.StringVar()
33
34 # Section générateur de mots de passe
35 tk.Label(root, text="Longueur du mot de passe :").pack()
36 entry_length = tk.Entry(root)
37 entry_length.pack()
38 tk.Button(root, text="Générer mot de passe", command=generate_password).pack()
39 tk.Label(root, textvariable=result_password, fg="blue").pack()
40

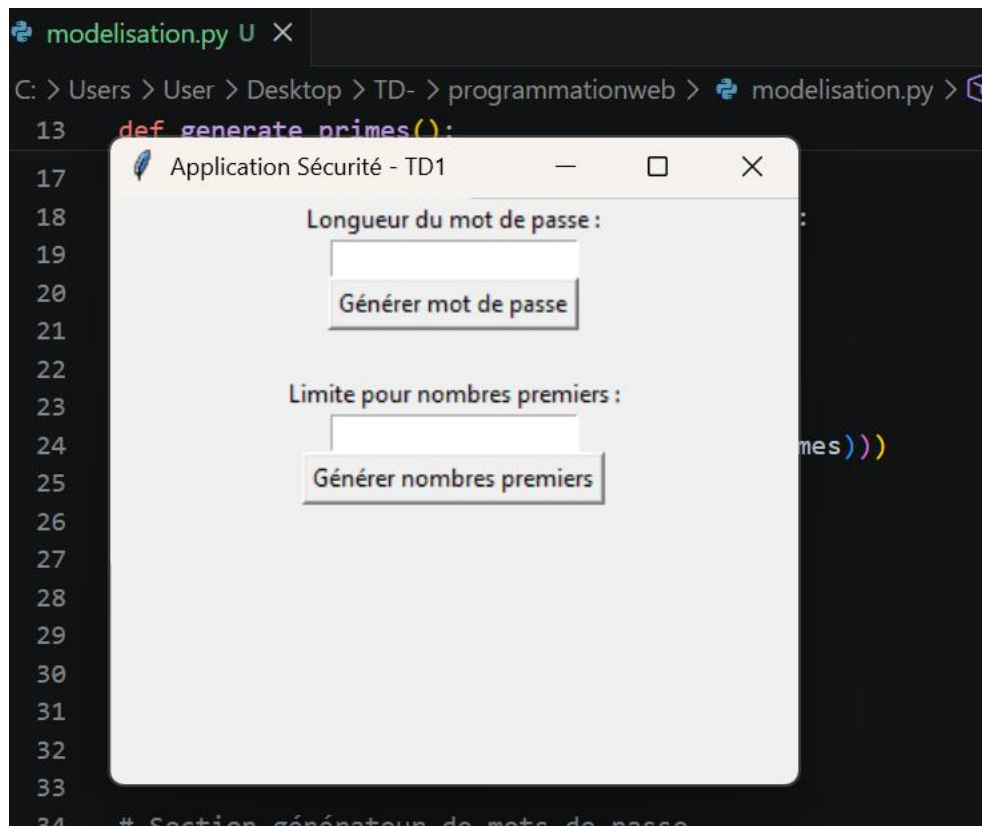
```



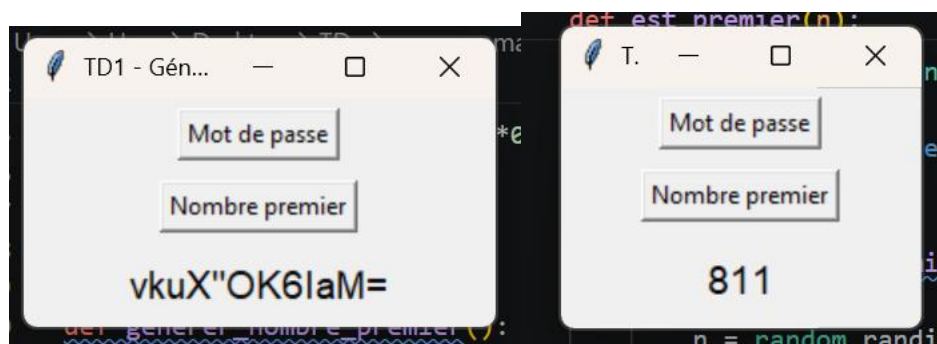
```

modelisation.py U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
39 tk.Label(root, textvariable=result_password, fg="blue").pack()
40
41 # Section générateur de nombres premiers
42 tk.Label(root, text="Limite pour nombres premiers :").pack()
43 entry_limit = tk.Entry(root)
44 entry_limit.pack()
45 tk.Button(root, text="Générer nombres premiers", command=generate_primes).pack()
46 tk.Label(root, textvariable=result_primes, fg="green", wraplength=400).pack()
47
48 # Lancer l'application
49 root.mainloop()
50

```



```
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python39-64/Python.exe C:/Users/User/Desktop/TD- > programmationweb/modelisation.py
✓ Nombre premier trouvé: 227
Résultat final : 227
PS C:\Users\User>
```



TD 2: Créer un programme Python qui :

- génère des nombres aléatoires
- teste leur primalité

- retourne un nombre premier
- Contraintes :
- utiliser une fonction de test de primalité
- afficher plusieurs essais

```

modelisation.py U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
1  import random
2
3  # Fonction de test de primalité
4  def is_prime(n):
5      if n < 2:
6          return False
7      for i in range(2, int(n**0.5) + 1):
8          if n % i == 0:
9              return False
10     return True
11
12 # Fonction principale : génère des nombres aléatoires et retourne un premier
13 def generate_prime(trials=10, limit=1000):
14     for attempt in range(trials):
15         num = random.randint(2, limit)
16         print(f"Essai {attempt+1}: nombre généré = {num}")
17         if is_prime(num):
18             print(f"✅ Nombre premier trouvé: {num}")
19             return num
20     print("❌ Aucun nombre premier trouvé dans les essais.")
21     return None
22
23 # =====
24 # 🟡 Exemple d'utilisation
25 # =====
26
27 prime_number = generate_prime(trials=15, limit=500)
28 print("Résultat final :", prime_number)
29

```



```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python
Essai 11: nombre généré = 159
Essai 12: nombre généré = 148
Essai 13: nombre généré = 315
Essai 14: nombre généré = 105
Essai 15: nombre généré = 219
❌ Aucun nombre premier trouvé dans les essais.
Résultat final : None
PS C:\Users\User>
```

TD 3 :Créer un programme Python qui demande à l'utilisateur de saisir deux nombres et calcule leur PGCD (Plus Grand CommunDiviseur) avec l'algorithme d'Euclide :

```
modelisation.py U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
1 # Fonction pour calculer le PGCD avec l'algorithme d'Euclide
2 def pgcd(a, b):
3     while b != 0:
4         a, b = b, a % b
5     return a
6
7 # Programme principal
8 print("=== Calcul du PGCD (Algorithme d'Euclide) ===")
9 x = int(input("Entrez le premier nombre : "))
10 y = int(input("Entrez le deuxième nombre : "))
11
12 resultat = pgcd(x, y)
13 print(f"Le PGCD de {x} et {y} est : {resultat}")
14

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Entrez le premier nombre : & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/modeli
...
rogrammationweb/modelisation.py'
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/modeli
=== Calcul du PGCD (Algorithme d'Euclide) ===
Entrez le premier nombre : 235
Entrez le deuxième nombre : 450
Le PGCD de 235 et 450 est : 5
```

```
rogrammationweb/modelisation.py'
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/
=== Calcul du PGCD (Algorithme d'Euclide) ===
Entrez le premier nombre : 235
Entrez le deuxième nombre : 450
Le PGCD de 235 et 450 est : 5
PS C:\Users\User> 
```